

Sri Lanka Institute of Information Technology



IT3060

Human Computer Interaction

Lab 5- Activity 2

Backend, Database and Authentication Comparison

IT23657878- W.R.S.D.Diwyanjali

Objective: Compare backend frameworks, database options and authentication/authorization solutions for a secure, scalable and maintainable FitFlow application.

1. FitFlow Requirements

The FitFlow case study describes a fitness application requiring AI-powered personalized workout plans, private social features, camera-based nutrition logging, progress dashboards, fast performance, offline capabilities and strong data security. The case study also selected Node.js with Express and Firebase for real-time social features and scalable notifications.

2. Backend Framework Comparison

Backend	Main Strengths	Main Limitations	FitFlow Suitability
Node.js / Express	Fast development, JavaScript ecosystem, real-time support	Single-threaded model needs careful handling of CPU-heavy workloads	Highly suitable
NestJS	Structured Node.js architecture, TypeScript, modular design	More framework structure and learning overhead than basic Express	Suitable for maintainable larger backend
Python / FastAPI	Fast API development, Python ecosystem, strong AI/ML integration	Python services may require separate architecture for some real-time/mobile workloads	Highly suitable for AI-heavy services
Go	High performance, concurrency, efficient resource usage	Smaller ecosystem for some application-level integrations; more development effort for some features	Suitable for performance-critical services

3. Database Comparison

Database	Main Strengths	Main Limitations	FitFlow Suitability
PostgreSQL	Relational structure, strong querying and transactions	Schema design can require more planning for rapidly changing data	Suitable for structured health and account data
MongoDB	Flexible document model, rapid development	Complex relational queries can be less natural than in relational databases	Suitable for flexible application data
Firebase Firestore	Real-time synchronization, easy mobile integration, scalable managed service	Complex querying/reporting can be less flexible and costs depend on usage	Highly suitable for real-time social features
DynamoDB	Highly scalable managed NoSQL	Requires careful access-pattern design	Suitable for high-scale cloud workloads

	database, low operational overhead		
--	------------------------------------	--	--

4. Authentication & Authorization Comparison

Authentication	Main Strengths	Main Limitations	FitFlow Suitability
Firebase Authentication	Easy integration, multiple sign-in methods, works well with Firebase	Strongly tied to Firebase ecosystem	Highly suitable
AWS Cognito	Managed authentication, scalable AWS integration	Can require more configuration and AWS knowledge	Suitable for AWS-based architecture
Auth0	Feature-rich identity and access management	Pricing and platform dependency need consideration	Suitable for advanced identity requirements
Supabase Auth	Simple developer experience and database integration	Best fit when using the wider Supabase ecosystem	Suitable for Supabase-centered systems

5. Overall Comparison

Option	Scalability	Security	Real-time	AI/ML Support	Cost Consideration
Node.js / Express	High	High	Strong	Strong	Moderate
Python / FastAPI	High	High	Strong	Very strong	Moderate
Go	Very high	Very high	Strong	Strong	Moderate
PostgreSQL	High	High	Strong	Strong	Moderate
MongoDB	High	High	Strong	Strong	Moderate
Firebase	High	High	Very strong	Strong	Low–Moderate
Firebase Auth	High	High	Strong	Good	Low–Moderate
AWS Cognito	High	High	Strong	Good	Moderate
Auth0	High	High	Strong	Good	Moderate–High

6. Recommended Combination

The recommended combination for the FitFlow redesign is Node.js with Express for the backend, Firebase/Firestore for real-time application data and Firebase Authentication for authentication.

Justification:

Node.js and Express support rapid API development and fit naturally with the React Native frontend selected in the FitFlow case study.

Firebase provides real-time capabilities required for social interactions and scalable notifications, which are explicitly mentioned in the case study.

Firebase Authentication provides a straightforward authentication layer that can integrate with the Firebase-based backend services.

The combination reduces the number of technologies that a mid-sized team must operate and maintain.

AI services can be integrated through backend APIs while keeping AI processing separate from the core application services when required.

Security controls, access rules and privacy protections can be applied around authentication, user data and real-time resources.

7. Security, Privacy and Maintainability

FitFlow handles wellness-related user information, so security and privacy must be considered throughout the architecture. Authentication should use secure identity management, authorization should restrict access to private social and user information, and data should be protected in transit and at rest. The case study also emphasizes strong data security and a privacy-focused architecture that minimizes unnecessary data transmission.

The selected combination is maintainable for a mid-sized team because it uses a JavaScript-based backend that aligns with the React Native ecosystem and managed Firebase services for real-time functionality and authentication.

8. Conclusion

For the FitFlow redesign, Node.js with Express, Firebase/Firestore and Firebase Authentication provide a coherent combination for rapid development, real-time social functionality, scalable notifications and manageable authentication. This recommendation is consistent with the technology stack documented in the FitFlow case study, which specifies Node.js with Express and Firebase for the backend and real-time features.

9. Source Basis

Primary basis: IT3060 Human Computer Interaction Semester 2 2026 FitFlow Redesign Case Study and Lab Exercise 05. The FitFlow-specific backend and Firebase selection follows the supplied case study; the comparison tables are structured to satisfy the Activity 2 requirements.